

SOLUTIONS - TP Final : Analyse Complète d'un Ransomware

Phase 1 : Reconnaissance Initiale - SOLUTIONS

1.1 Calcul des Hashs

Commandes :

```
md5sum cryptolocker_advanced.py
sha1sum cryptolocker_advanced.py
sha256sum cryptolocker_advanced.py
```

Réponses :

- **Q1** : Les hashs varient selon le système, mais voici un exemple :
 - MD5: `a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6`
 - SHA1: `1a2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0`
 - SHA256: `a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6`
- **Q2** : Ces hashs peuvent être :
 - Soumis à VirusTotal pour vérifier si le malware est connu
 - Utilisés dans des règles YARA/Snort pour la détection
 - Partagés avec d'autres analystes (IOC sharing)
 - Stockés dans des bases de données de threat intelligence
 - Utilisés pour vérifier l'intégrité de l'échantillon

1.2 Analyse des Métadonnées

Réponses :

- **Q3** : Python script, ASCII text executable
- **Q4** : Environ 250-300 lignes (dépend du comptage des lignes vides)

1.3 Analyse des Imports

Imports détectés :

```
import os
import sys
import base64
import hashlib
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
```

```
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2
import socket
import subprocess
import time
from pathlib import Path
```

Réponses :

- **Q5** : Imports cryptographiques suspects :
 - **cryptography.fernet** : Chiffrement symétrique Fernet (AES-128 en mode CBC)
 - **cryptography.hazmat.primitives.kdf.pbkdf2** : Dérivation de clé PBKDF2
 - **hashlib** : Fonctions de hachage (MD5, SHA256)
- **Q6** : Imports réseau :
 - **socket** : Communications TCP/IP brutes
- **Q7** : Imports suspects pour un ransomware :
 - **cryptography** : Chiffrement de fichiers
 - **socket** : Communication C2
 - **subprocess** : Exécution de commandes système (suppression de backups)
 - **base64** : Obfuscation de configuration
 - **os** : Manipulation de fichiers et système de fichiers

Phase 2 : Analyse Statique Approfondie - SOLUTIONS

2.1 Extraction des Chaînes

Réponses :

- **Q8** : Adresse Bitcoin : **1BoatSLRHtKNngkdXEeobR76b53LEtTpyT**
 - Note : Adresse fictive générée pour l'exercice
- **Q9** : Montant : **0.75 BTC** (Bitcoin)
 - À titre indicatif, cela représente environ \$30,000-\$50,000 selon le cours
- **Q10** : Délai : **72 heures** (3 jours)
 - Technique psychologique classique créant urgence et stress

2.2 Désobfuscation des Variables

Script de désobfuscation :

```
#!/usr/bin/env python3
import base64
```

```
# Variable _0x1a2b : Configuration C2
encoded_c2 = b'MTkyLjE2OC4xLjEwMDo4MDgw'
decoded_c2 = base64.b64decode(encoded_c2).decode()
print(f"C2 Server: {decoded_c2}")

# Résultat : 192.168.1.100:8080
```

Réponses :

- **Q11** : `_0x1a2b` contient : `192.168.1.100:8080`
 - Serveur C2 (Command & Control)
 - IP privée (RFC 1918) = fictive pour l'exercice

- **Q12** : `_0x3c4d` contient :

```
['.txt', '.pdf', '.docx', '.xlsx', '.jpg', '.png', '.zip']
```

- Extensions de fichiers ciblés
- Cible les documents et images (données précieuses pour les victimes)

- **Q13** : `_0x5e6f` contient :

```
b'SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE'
```

- **Sel cryptographique** pour PBKDF2
- Utilisé pour dériver la clé de chiffrement
- Hardcodé = faiblesse critique (expliqué plus bas)

2.3 Analyse des Méthodes

Tableau de correspondance :

Méthode	Fonction	Description Technique
<code>_0x1()</code>	Mutex Check	Crée un fichier verrou <code>/tmp/.crypto_<HASH></code> pour empêcher les exécutions multiples
<code>_0x2()</code>	Generate Victim ID	SHA256(username@hostname) encodé en Base64, tronqué à 32 caractères
<code>_0x3()</code>	Derive Encryption Key	PBKDF2(victim_id, salt, 100000 iterations, SHA256) → clé Fernet
<code>_0x4()</code>	Install Persistence	Copie dans <code>~/.system/WinDefender/</code> + <code>.desktop</code> dans autostart
<code>_0x5()</code>	Contact C2	Socket TCP vers 192.168.1.100:8080, envoie <code>VICTIM:<ID></code>
<code>_0x6()</code>	Encrypt File	Fernet.encrypt() → écrit <code>.locked</code> , supprime l'original

Méthode	Fonction	Description Technique
<code>_0x7()</code>	Scan & Encrypt	<code>os.walk()</code> parcourt récursivement, limite à 15 fichiers
<code>_0x8()</code>	Drop Ransom Note	Écrit <code>HOW_TO_DECRYPT.txt</code> avec instructions
<code>_0x9()</code>	Delete Shadow Copies	Simulation : log des commandes <code>vssadmin/bcdedit/wbadmin</code>

Réponses détaillées :

- **Q14 : `_0x1()` - Mutex (Mutual Exclusion)**
 - Objectif : Empêcher plusieurs instances du ransomware de s'exécuter simultanément
 - Méthode : Crée `/tmp/.crypto_<MD5_HASH>`
 - Raison : Éviter les conflits (double chiffrement = perte de données)
- **Q15 : `_0x2()` - Génération d'Identifiant Victime**
 - Formule : `SHA256(username@hostname)` → Base64 → 32 premiers caractères
 - Exemple : `dXNlckBob3N0bmFtZQ==` devient `dXNlckBob3N0bmFtZQ==abcd1234...`
 - Utilité : Identifier la victime auprès du serveur C2, générer la clé
- **Q16 : `_0x3()` - Dérivation de Clé de Chiffrement**
 - Algorithme : **PBKDF2-HMAC-SHA256**
 - Paramètres :
 - Input : Victim ID (32 chars)
 - Salt : `SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE`
 - Iterations : **100,000**
 - Output : 32 bytes → Base64 URL-safe (clé Fernet)
 - **FAILLE CRITIQUE** : Le sel est hardcodé dans le code source !
 - Conséquence : Si on connaît le victim ID, on peut régénérer la clé
 - Un ransomware professionnel utiliserait un sel unique par victime
- **Q17 : `_0x4()` - Installation de la Persistance**
 - Chemin installation : `~/system/WinDefender/defender.py`
 - Mécanisme : XDG Autostart (Linux Desktop)
 - Fichier : `~/config/autostart/WindowsDefenderUpdate.desktop`
 - Mimicry : Nom "Windows Defender" pour tromper les utilisateurs Linux naïfs
- **Q18 : `_0x5()` - Communication C2**
 - Protocole : TCP brut (non HTTP, contrairement au TP2)
 - Payload : `VICTIM:<32-char-id>\n`
 - Timeout : 5 secondes
 - Réponse attendue : Instructions de paiement ou clé de déchiffrement
- **Q19 : `_0x6()` - Chiffrement d'un Fichier**
 - Lecture du contenu original

- `Fernet.encrypt(data)` → Chiffrement AES-128 CBC + HMAC
- Écriture dans `<original>.locked`
- **Suppression de l'original** (`os.remove()`)
- **Q20 : `_0x7()` - Parcours et Chiffrement**
 - `os.walk()` : Parcours récursif de l'arbre de répertoires
 - Filtres :
 - Extensions : `.txt`, `.pdf`, `.docx`, `.xlsx`, `.jpg`, `.png`, `.zip`
 - Exclusions : `.system`, `.config` (pour ne pas casser la persistance)
 - **Limite éducative** : 15 fichiers maximum
- **Q21 : `_0x8()` - Note de Rançon**
 - Nom : `HOW_TO_DECRYPT.txt`
 - Contenu :
 - Instructions de paiement
 - Adresse Bitcoin
 - Victim ID
 - Menaces (deadline, perte permanente)
 - Conseils de sécurité (ironique)
- **Q22 : `_0x9()` - Suppression des Backups**
 - **Sur Windows (simulation) :**

```
vssadmin delete shadows /all /quiet  
bcdedit /set {default} recoveryenabled no  
wbadmin delete catalog -quiet
```
 - **Sur Linux** : Pas d'équivalent direct (snapshots LVM, Timeshift)
 - Objectif : Empêcher la restauration depuis backups système

2.4 Analyse de l'Algorithme de Chiffrement

Réponses :

- **Q23 : Algorithme : Fernet (cryptography library)**
 - Spécifications techniques :
 - Chiffrement : **AES-128** en mode **CBC**
 - Authentification : **HMAC-SHA256**
 - Format : Base64(version || timestamp || IV || ciphertext || HMAC)
 - Référence : <https://github.com/fernet/spec/>
- **Q24 : Dérivation de la clé :**

```
PBKDF2-HMAC-SHA256(  
    password = victim_id,
```

```
    salt = "SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE",
    iterations = 100000,
    dkLen = 32
) → Base64-URLSafe → Clé Fernet
```

- **Q25** : Nombre d'itérations : **100,000**
 - Standard moderne (OWASP recommande $\geq 310,000$ pour SHA256)
 - Rend le bruteforce plus coûteux (mais pas infaisable)
- **Q26** : Sel : **SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE**
 - **Faible majeure** : Sel hardcodé et identique pour toutes les victimes
 - Conséquence : Rainbow tables possibles si le victim ID est faible

Phase 3 : Analyse Dynamique - SOLUTIONS

3.1 Préparation de l'Environnement

Vérification :

```
ls -la ~/Documents/VictimFiles/
# Doit contenir doc_1.txt ... doc_5.txt, photo_1.jpg ... photo_5.jpg
```

3.2 Traçage des Appels Système

Commande complète :

```
strace -f -e trace=openat,unlink,socket,connect,write -o strace_ransom.log \
python3 cryptolocker_advanced.py 2>&1 | tee execution.log
```

Analyse du fichier **strace_ransom.log :**

Réponses :

- **Q27** : Fichiers ouverts en lecture (**O_RDONLY**) :

```
grep "openat.*O_RDONLY" strace_ransom.log | grep -v ".pyc\|site-packages"
```

Résultats attendus :

- **~/Documents/VictimFiles/doc_1.txt**
- **~/Documents/VictimFiles/doc_2.txt**

- ~/Documents/VictimFiles/photo_1.jpg
- etc.

• **Q28** : Fichiers supprimés (unlink) :

```
grep "unlink" strace_ransom.log
```

Résultats attendus :

- unlink("/home/user/Documents/VictimFiles/doc_1.txt")
- unlink("/home/user/Documents/VictimFiles/doc_2.txt")
- etc.
- **Preuve de destruction** : Les originaux sont détruits après chiffrement

• **Q29** : Fichiers créés (O_CREAT) :

```
grep "openat.*O_CREAT" strace_ransom.log | grep -v ".pyc"
```

Résultats attendus :

- /tmp/.crypto_<hash> (mutex)
- ~/.system/WinDefender/defender.py (copie de persistance)
- ~/.config/autostart/WindowsDefenderUpdate.desktop
- ~/Documents/VictimFiles/doc_1.txt.locked
- ~/Documents/VictimFiles/HOW_TO_DECRYPT.txt
- /tmp/.shadow_delete.log

• **Q30** : Connexion réseau :

```
grep "connect" strace_ransom.log
```

Résultat attendu :

```
connect(3, {sa_family=AF_INET, sin_port=htons(8080),  
sin_addr=inet_addr("192.168.1.100")}, 16) = -1 EHOSTUNREACH (No route  
to host)
```

- **Échec de connexion** : IP fictive, pas de serveur C2 réel
- Preuve de tentative de beacon

3.3 Surveillance en Temps Réel des Fichiers

Commande :

```
inotifywait -m -r ~/Documents/VictimFiles/ \
-e create,delete,modify,moved_to,moved_from \
2>&1 | tee inotify_ransom.log
```

Événements détectés :

Réponses :

- **Q31** : Événements inotify :

```
/home/user/Documents/VictimFiles/ OPEN doc_1.txt
/home/user/Documents/VictimFiles/ ACCESS doc_1.txt
/home/user/Documents/VictimFiles/ CLOSE_NOWRITE,CLOSE doc_1.txt
/home/user/Documents/VictimFiles/ CREATE doc_1.txt.locked
/home/user/Documents/VictimFiles/ OPEN doc_1.txt.locked
/home/user/Documents/VictimFiles/ MODIFY doc_1.txt.locked
/home/user/Documents/VictimFiles/ CLOSE_WRITE,CLOSE doc_1.txt.locked
/home/user/Documents/VictimFiles/ DELETE doc_1.txt
```

- **Q32** : Ordre des opérations :

1. **OPEN + ACCESS** : Lecture du fichier original
2. **CREATE** : Création du fichier **.locked**
3. **MODIFY** : Écriture du contenu chiffré
4. **DELETE** : Suppression de l'original

- **Q33** : Extension ajoutée : **.locked**
 - Indicateur visuel pour la victime
 - Facilite l'identification des fichiers chiffrés pour le déchiffrement

3.4 Capture du Trafic Réseau

Commande :

```
sudo tcpdump -i any -l -A -s 0 'host 192.168.1.100 and port 8080' \
2>&1 | tee tcpdump_ransom.log &
```

Analyse du trafic :

Réponses :

- **Q34** : Adresse IP contactée : **192.168.1.100**
 - IP privée (RFC 1918 : 192.168.0.0/16)
 - Fictive pour l'exercice

- **Q35** : Port : **8080**
 - Port HTTP alternatif commun
 - Non privilégié (< 1024)

- **Q36** : Données envoyées :

```
VICTIM:dXNlckBob3N0bmFtZQ==abcd1234
```

- Format : **VICTIM:<32-char-Base64-ID>**
 - Exemple : **VICTIM:dXNlckBob3N0bmFtZQ==abcd1234**
- **Q37** : Réponse du serveur :
 - **Aucune** (dans l'exercice standard)
 - Raison : IP fictive, pas de serveur C2 réel
 - Dans un scénario réel, réponse attendue :

```
OK:ENCRYPTED  
ou  
KEY:<base64-decryption-key>
```

Si vous souhaitez simuler un serveur C2, créez un script Python :

```
#!/usr/bin/env python3  
import socket  
  
HOST = '0.0.0.0'  
PORT = 8080  
  
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  
    s.bind((HOST, PORT))  
    s.listen()  
    print(f"[*] C2 Server listening on {HOST}:{PORT}")  
  
    while True:  
        conn, addr = s.accept()  
        with conn:  
            print(f"[+] Connection from {addr}")  
            data = conn.recv(1024).decode()  
            print(f"[>] Received: {data}")  
  
            response = "OK:ENCRYPTED\n"  
            conn.send(response.encode())  
            print(f"[<] Sent: {response}")
```

Modifiez ensuite le ransomware pour pointer vers **127.0.0.1:8080**.

Phase 4 : Extraction des IOCs - SOLUTIONS

4.1 Checklist Complète des IOCs

IOCs Réseau

- ☒ Adresse IP C2 : **192.168.1.100**
- ☒ Port C2 : **8080**
- ☒ Protocole : **TCP brut** (non HTTP)
- ☒ Format des données envoyées : **VICTIM:<32-char-Base64-ID>\n**

IOCs Fichiers

- ☒ Mutex : **Global{C7D8E9F0-A1B2-C3D4-E5F6-071829384756}**
 - Fichier : **/tmp/.crypto_c7d8e9f0a1b2c3d4e5f6071829384756**
- ☒ Chemin d'installation : **~/system/WinDefender/**
- ☒ Nom du fichier de persistance : **defender.py**
- ☒ Extension des fichiers chiffrés : **.locked**
- ☒ Nom de la note de rançon : **HOW_TO_DECRYPT.txt**
- ☒ Fichiers temporaires créés : **/tmp/.shadow_delete.log**

IOCs Cryptographiques

- ☒ Algorithme de chiffrement : **Fernet (AES-128-CBC + HMAC-SHA256)**
- ☒ Algorithme de dérivation de clé : **PBKDF2-HMAC-SHA256**
- ☒ Sel utilisé : **SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE**
- ☒ Nombre d'itérations PBKDF2 : **100,000**

IOCs Comportementaux

- ☒ Extensions de fichiers ciblées : **['.txt', '.pdf', '.docx', '.xlsx', '.jpg', '.png', '.zip']**
- ☒ Limite de fichiers chiffrés : **15** (éducatif)
- ☒ Commandes de suppression de backups :

```
vssadmin delete shadows /all /quiet
bcdedit /set {default} recoveryenabled no
wbadmin delete catalog -quiet
```

IOCs Financiers

- ☒ Adresse Bitcoin : **1BoatSLRHtKNngkdXEeobR76b53LEtptyT**
- ☒ Montant demandé : **0.75 BTC**
- ☒ Email de contact : **recovery@cryptolocker-edu.com**
- ☒ Délai de paiement : **72 heures**

4.2 Règle YARA Complète

```
rule CryptoLocker_Educational {
  meta:
    description = "Détection le ransomware CryptoLocker éducatif"
    author = "Cyber Security Lab"
    date = "2024-12-19"
    threat_level = "high"
    malware_type = "ransomware"

  strings:
    // IOCs Financiers
    $bitcoin = "1BoatSLRHtKNngkdXEeobR76b53LETtpyT" ascii
    $ransom_amount = "0.75 BTC" ascii
    $email = "recovery@cryptolocker-edu.com" ascii

    // IOCs Fichiers
    $mutex = "Global\\{C7D8E9F0-A1B2-C3D4-E5F6-071829384756}" ascii
    $persistence_path = ".system/WinDefender" ascii
    $persistence_name = "WindowsDefenderUpdate" ascii
    $ransom_note = "HOW_TO_DECRYPT.txt" ascii
    $locked_ext = ".locked" ascii

    // IOCs Réseau (Base64 encodé)
    $c2_encoded = "MTkyLjE2OC4xLjEwMDo4MDgw" ascii //
192.168.1.100:8080

    // IOCs Cryptographiques
    $salt = "SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE" ascii
    $fernet_import = "from cryptography.fernet import Fernet" ascii
    $pbkdf2_import = "from cryptography.hazmat.primitives.kdf.pbkdf2
import PBKDF2" ascii

    // IOCs Comportementaux
    $extensions = "['.txt', '.pdf', '.docx', '.xlsx', '.jpg', '.png',
'.zip']" ascii
    $victim_prefix = "VICTIM:" ascii

    // Commandes de suppression de backups
    $vss_delete = "vssadmin delete shadows /all /quiet" ascii
    $bcdedit = "bcdedit /set {default} recoveryenabled no" ascii
    $wbadmin = "wbadmin delete catalog -quiet" ascii

  condition:
    // Au moins 3 IOCs financiers
    ($bitcoin or $ransom_amount or $email) and

    // Au moins 2 IOCs de persistance
    ($mutex or $persistence_path or $persistence_name) and

    // Au moins 1 IOC cryptographique
    ($salt or $fernet_import or $pbkdf2_import) and
```

```
// Note de rançon ou extension chiffrée
($ransom_note or $locked_ext) and

// Taille du fichier raisonnable pour un script Python
filesize < 500KB

}
```

Test de la règle :

```
yara cryptolocker.yar cryptolocker_advanced.py
# Doit afficher : CryptoLocker_Educational cryptolocker_advanced.py
```

4.3 Règle Snort Complète

```
# Détection de la communication C2 du ransomware CryptoLocker

alert tcp $HOME_NET any -> 192.168.1.100 8080 (
  msg:"MALWARE CryptoLocker C2 Beacon Detected";
  content:"VICTIM:";
  depth:7;
  content:"|0a|";
  within:50;
  classtype:trojan-activity;
  sid:1000002;
  rev:2;
  metadata:created_at 2024-12-19, updated_at 2024-12-19;
  reference:url,github.com/your-lab/cryptolocker-analysis;
)

alert tcp 192.168.1.100 8080 -> $HOME_NET any (
  msg:"MALWARE CryptoLocker C2 Response Detected";
  content:"OK:ENCRYPTED";
  depth:20;
  flow:established,from_server;
  classtype:trojan-activity;
  sid:1000003;
  rev:1;
  metadata:created_at 2024-12-19;
)
```

Explications :

- **content:"VICTIM:"** : Recherche de la signature du beacon
- **depth:7** : Les 7 premiers bytes doivent contenir "VICTIM:"
- **content:"|0a|"** : Recherche d'un line feed (n) dans les 50 bytes suivants
- **flow:established,from_server** : Uniquement les réponses du serveur

Phase 5 : Remédiation et Prévention - SOLUTIONS

5.1 Supprimer la Persistance

Commandes complètes :

```
# Identifier la persistance
ls -la ~/.config/autostart/ | grep -i "defender\|windows"
ls -la ~/.system/

# Supprimer l'autostart
rm -v ~/.config/autostart/WindowsDefenderUpdate.desktop

# Supprimer l'installation
rm -rfv ~/.system/WinDefender/

# Supprimer le mutex
rm -v /tmp/.crypto_*

# Vérifier
ps aux | grep defender
crontab -l | grep defender
```

5.2 Analyse de la Possibilité de Déchiffrement

Réponses :

- **Q38** : Est-il possible de déchiffrer sans payer ?
 - **OUI**, dans ce cas éducatif, car :
 1. Le sel est hardcodé dans le code source
 2. Le victim ID est dérivé de `username@hostname` (prévisible)
 3. Pas de composant aléatoire dans la génération de clé
 4. Fernet n'utilise pas de clé asymétrique
 - **NON**, dans un ransomware professionnel, car :
 1. Utilise RSA ou ECC (chiffrement asymétrique)
 2. La clé privée est uniquement sur le serveur C2
 3. Chaque victime a une paire de clés unique
 4. Impossible de régénérer la clé sans l'aide des attaquants
- **Q39** : Informations nécessaires pour déchiffrer :
 1. **Victim ID** (32 caractères Base64)
 2. **Sel** (hardcodé dans le code : `SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE`)
 3. **Nombre d'itérations PBKDF2** (100,000)
 4. **Algorithme** (PBKDF2-HMAC-SHA256)

Avec ces informations, on peut régénérer la clé Fernet.

- **Q40** : Où trouver le victim ID ?
 - **Option 1** : Dans la note de rançon `HOW_TO_DECRYPT.txt`
 - **Option 2** : Régénérer avec le script :

```
import os
import socket
import hashlib
import base64

username = os.getlogin()
hostname = socket.gethostname()
combined = f"{username}@{hostname}".encode()
victim_id =
base64.b64encode(hashlib.sha256(combined).digest()).decode()[ :32]
print(f"Victim ID: {victim_id}")
```

5.3 Script de Déchiffrement Complet

```
#!/usr/bin/env python3
"""
Decryptor for CryptoLocker Educational Ransomware
"""

import os
import sys
import base64
from pathlib import Path
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2

def derive_key(victim_id):
    """
    Dérive la clé de chiffrement à partir du victim ID
    Utilise les mêmes paramètres que le ransomware
    """
    _salt = b'SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE'
    _kdf = PBKDF2(
        algorithm=hashes.SHA256(),
        length=32,
        salt=_salt,
        iterations=100000,
    )
    return base64.urlsafe_b64encode(_kdf.derive(victim_id.encode()))

def decrypt_file(encrypted_path, key):
    """
    Déchiffre un fichier .locked
    """
```

```
try:
    # Vérifier que le fichier existe
    if not os.path.exists(encrypted_path):
        print(f"[-] File not found: {encrypted_path}")
        return False

    # Vérifier l'extension
    if not encrypted_path.endswith('.locked'):
        print(f"[-] Not a .locked file: {encrypted_path}")
        return False

    # Lire le contenu chiffré
    with open(encrypted_path, 'rb') as f:
        encrypted_data = f.read()

    # Déchiffrer
    cipher = Fernet(key)
    decrypted_data = cipher.decrypt(encrypted_data)

    # Écrire le fichier déchiffré (sans .locked)
    original_path = encrypted_path[:-7] # Retirer ".locked"
    with open(original_path, 'wb') as f:
        f.write(decrypted_data)

    # Supprimer le fichier chiffré
    os.remove(encrypted_path)

    print(f"[+] Decrypted: {original_path}")
    return True

except Exception as e:
    print(f"[-] Error decrypting {encrypted_path}: {e}")
    return False

def find_locked_files(directory):
    """
    Trouve tous les fichiers .locked dans un répertoire
    """
    locked_files = []
    for root, dirs, files in os.walk(directory):
        for file in files:
            if file.endswith('.locked'):
                locked_files.append(os.path.join(root, file))
    return locked_files

def main():
    print("=" * 60)
    print("CryptoLocker Educational Ransomware - Decryptor")
    print("=" * 60)

    if len(sys.argv) != 3:
        print("\nUsage: python3 decrypt.py <VICTIM_ID> <TARGET_DIRECTORY>")
        print("\nExample:")
        print("  python3 decrypt.py dXNlckBob3N0bmFtZQ==abcd1234")
```

```
~/Documents/VictimFiles")
    sys.exit(1)

victim_id = sys.argv[1]
target_dir = sys.argv[2]

# Vérifier que le répertoire existe
if not os.path.isdir(target_dir):
    print(f"[-] Directory not found: {target_dir}")
    sys.exit(1)

# Dériver la clé
print(f"\n[*] Deriving decryption key from Victim ID: {victim_id}")
key = derive_key(victim_id)
print(f"[*] Key derived: {key[:20].decode()}...")

# Trouver les fichiers chiffrés
print(f"\n[*] Scanning for .locked files in: {target_dir}")
locked_files = find_locked_files(target_dir)

if not locked_files:
    print("[-] No .locked files found!")
    sys.exit(0)

print(f"[+] Found {len(locked_files)} encrypted files")

# Demander confirmation
print("\n[!] Ready to decrypt. Continue? (y/n): ", end='')
if input().lower() != 'y':
    print("[-] Aborted")
    sys.exit(0)

# Déchiffrer tous les fichiers
print("\n[*] Starting decryption...\n")
success_count = 0
fail_count = 0

for encrypted_file in locked_files:
    if decrypt_file(encrypted_file, key):
        success_count += 1
    else:
        fail_count += 1

# Résumé
print("\n" + "=" * 60)
print(f"Decryption complete!")
print(f"  Success: {success_count}")
print(f"  Failed:  {fail_count}")
print("=" * 60)

# Supprimer la note de rançon
ransom_note = os.path.join(target_dir, "HOW_TO_DECRYPT.txt")
if os.path.exists(ransom_note):
    os.remove(ransom_note)
```



```
print(f"\n[+] Removed ransom note: {ransom_note}")

if __name__ == "__main__":
    main()
```

Utilisation :

```
# Obtenir le victim ID
grep "YOUR UNIQUE VICTIM ID" ~/Documents/VictimFiles/HOW_TO_DECRYPT.txt

# Déchiffrer
python3 decrypt.py <VICTIM_ID> ~/Documents/VictimFiles

# Exemple
python3 decrypt.py dXNlckBob3N0bmFtZQ==abcd1234 ~/Documents/VictimFiles
```

Sortie attendue :

```
=====
CryptoLocker Educational Ransomware - Decryptor
=====

[*] Deriving decryption key from Victim ID: dXNlckBob3N0bmFtZQ==abcd1234
[*] Key derived: Z0FBQUFBQm5kWGsYm1G...

[*] Scanning for .locked files in: /home/user/Documents/VictimFiles
[+] Found 15 encrypted files

[!] Ready to decrypt. Continue? (y/n): y

[*] Starting decryption...

[+] Decrypted: /home/user/Documents/VictimFiles/doc_1.txt
[+] Decrypted: /home/user/Documents/VictimFiles/doc_2.txt
[+] Decrypted: /home/user/Documents/VictimFiles/photo_1.jpg
...

=====
Decryption complete!
  Success: 15
  Failed:  0
=====

[+] Removed ransom note:
/home/user/Documents/VictimFiles/HOW_TO_DECRYPT.txt
```

A. Pourquoi le Déchiffrement est Possible ?

Faible 1 : Sel Hardcodé

```
_0x5e6f = b'SALT_FOR_KEY_DERIVATION_DO_NOT_SHARE'
```

- Le sel est **identique pour toutes les victimes**
- Un ransomware professionnel générerait un sel aléatoire par victime
- Sans sel unique, on peut pré-calculer des rainbow tables

Faible 2 : Clé Déterministe

```
self._victim_id =
base64.b64encode(hashlib.sha256(combined).digest()).decode()[ :32]
```

- La clé dépend uniquement de `username@hostname`
- Aucune composante aléatoire (pas de timestamp, pas de nonce)
- Prévisible si on connaît l'environnement de la victime

Faible 3 : Chiffrement Symétrique Uniquement

- Fernet = AES-128 (symétrique)
- Même clé pour chiffrement et déchiffrement
- Un ransomware professionnel utilise **RSA + AES** :
 1. Génère une clé AES aléatoire par victime
 2. Chiffre les fichiers avec cette clé AES
 3. Chiffre la clé AES avec la clé publique RSA
 4. Seule la clé privée RSA (sur le serveur C2) peut déchiffrer

B. Architecture d'un Ransomware Professionnel

[Victime]	[Serveur C2]
1. Génère paire RSA locale	
(publique/privée)	
2. Génère clé AES aléatoire	
3. Chiffre fichiers avec AES	
4. Chiffre clé AES avec RSA public	
5. Supprime clé AES originale	
6. Envoie clé RSA publique ----->	

```

|         (clé privée RSA reste sur C2)         |
| 7. Affiche note de rançon                     |
| <--- Après paiement, reçoit clé privée RSA   |
| 8. Déchiffre clé AES                         |
| 9. Déchiffre fichiers                       |

```

C. Comparaison des Techniques

Aspect	CryptoLocker Educational	Ransomware Professionnel
Chiffrement	Fernet (AES-128 symétrique)	RSA-2048 + AES-256
Dérivation de clé	PBKDF2 avec sel hardcodé	Clé AES aléatoire + RSA
Déchiffrement sans payer	✓ Possible (sel connu)	✗ Impossible (clé privée sur C2)
Obfuscation	Base64 simple	Polymorphisme, packing, anti-debug
Persistance	XDG Autostart	Multiples (Registry, Services, Scheduled Tasks)
Communication C2	TCP brut	HTTPS, Tor, Domain Fronting
Suppression backups	Simulé	Réel (vssadmin, wbadmin)
Propagation	Aucune	Latérale (SMB, RDP, exploits)

D. Amélioration du Ransomware (Perspective Défensive)

Ce qu'un attaquant sophistiqué ajouterait :

1. Chiffrement hybride RSA+AES :

```

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes
import os

# Génération clé AES aléatoire
aes_key = os.urandom(32)

# Chiffrement des fichiers avec AES
cipher = Cipher(algorithms.AES(aes_key), modes.CFB(os.urandom(16)))

# Chiffrement de la clé AES avec RSA (clé publique hardcodée)

```

```
public_key = load_public_key()
encrypted_aes_key = public_key.encrypt(aes_key, padding.OAEP(...))

# Suppression de la clé AES en clair
del aes_key
```

2. Obfuscation avancée :

- PyArmor pour obfusquer le bytecode Python
- PyInstaller pour compiler en binaire
- UPX pour compresser/packer

3. Anti-analyse :

```
# Détection de VM/sandbox
if is_vm() or is_sandbox():
    sys.exit(0)

# Détection de debugger
if is_debugger_present():
    os.remove(__file__)
    sys.exit(0)
```

4. Propagation latérale :

```
# Scan réseau local
for ip in network_scan():
    try:
        exploit_smb(ip)
        upload_ransomware(ip)
    except:
        continue
```

5. Communication C2 anonyme :

- Tor Hidden Services (.onion)
- Domain Fronting (CloudFlare, AWS)
- DNS Tunneling

Scénarios de Remédiation

Scénario 1 : Détection Précoce

Indicateurs de détection :

- Activité inhabituelle de lecture/écriture de fichiers
- Connexions réseau vers IPs suspectes

- Création de fichiers .locked
- Modifications de l'autostart

Actions immédiates :

```
# Isoler la machine
sudo iptables -A OUTPUT -j DROP
sudo systemctl stop NetworkManager

# Tuer le processus
ps aux | grep "defender\|cryptolocker"
sudo kill -9 <PID>

# Préserver les preuves
sudo dd if=/dev/sda of=/mnt/external/disk_image.dd bs=4M
```

Scénario 2 : Récupération Post-Infection

Si backups disponibles :

```
# Restaurer depuis backup
rsync -av /mnt/backup/Documents/ ~/Documents/

# Vérifier l'intégrité
sha256sum -c checksums.txt
```

Si pas de backups :

1. Identifier le victim ID
2. Utiliser le script de déchiffrement
3. Vérifier l'intégrité des fichiers récupérés

Scénario 3 : Analyse Forensique

Collection d'artefacts :

```
# Mémoire RAM
sudo apt install lime-forensics-dkms
sudo insmod /path/to/lime.ko "path=/tmp/memory.lime format=lime"

# Logs système
journalctl --since "2024-12-19 08:00:00" > /tmp/system.log
cat /var/log/syslog | grep -i "defender\|cryptolocker" > /tmp/malware.log

# Timeline
sudo apt install sleuthkit
fls -r /dev/sda1 > /tmp/filesystem_timeline.txt
```

Annexe : IOCs Machine-Readable

Format STIX 2.1

```
{
  "type": "bundle",
  "id": "bundle--cryptolocker-edu",
  "objects": [
    {
      "type": "malware",
      "id": "malware--cryptolocker-edu",
      "name": "CryptoLocker Educational",
      "malware_types": ["ransomware"],
      "is_family": false
    },
    {
      "type": "indicator",
      "id": "indicator--bitcoin-addr",
      "pattern": "[file:content_ref.payload_bin matches '1BoatSLRhtKNgkdXEeobR76b53LETtpyT']",
      "valid_from": "2024-12-19T00:00:00Z"
    },
    {
      "type": "indicator",
      "id": "indicator--c2-ip",
      "pattern": "[network-traffic:dst_ref.type = 'ipv4-addr' AND network-traffic:dst_ref.value = '192.168.1.100' AND network-traffic:dst_port = 8080]",
      "valid_from": "2024-12-19T00:00:00Z"
    }
  ]
}
```

Format OpenIOC

```
<?xml version="1.0" encoding="UTF-8"?>
<ioc xmlns="http://schemas.mandiant.com/2010/ioc">
  <short_description>CryptoLocker Educational Ransomware</short_description>
  <definition>
    <Indicator operator="OR">
      <IndicatorItem>
        <Context document="FileItem" search="FileItem/FileName"/>
        <Content type="string">HOW_TO_DECRYPT.txt</Content>
      </IndicatorItem>
      <IndicatorItem>
        <Context document="FileItem" search="FileItem/FileName"/>
        <Content type="string">.locked</Content>
      </IndicatorItem>
    </Indicator>
  </definition>
</ioc>
```

```
<IndicatorItem>
  <Context document="NetworkItem" search="NetworkItem/RemoteIP"/>
  <Content type="IP">192.168.1.100</Content>
</IndicatorItem>
</Indicator>
</definition>
</ioc>
```

BONUS : Test avec Serveur C2 Fonctionnel

Objectif

Tester le ransomware avec un vrai serveur C2 local pour observer la communication complète.

Étape 1 : Démarrer le Serveur C2

Le répertoire contient un simulateur de serveur C2 : `fake_c2_server.py`

Terminal 1 :

```
cd /root/Cours/RansomwareRE
python3 fake_c2_server.py
```

Sortie attendue :

```
[2024-12-11 21:44:06]
=====
[2024-12-11 21:44:06] CryptoLocker Educational - C2 Server Simulator
[2024-12-11 21:44:06]
=====
[2024-12-11 21:44:06] [*] Starting C2 server on 0.0.0.0:8080
[2024-12-11 21:44:06] [*] C2 Server listening on 0.0.0.0:8080
[2024-12-11 21:44:06] [*] Press Ctrl+C to stop
[2024-12-11 21:44:06]
=====
```

Étape 2 : Utiliser la Version Test du Ransomware

Le fichier `cryptolocker_test_c2.py` est configuré pour se connecter à `localhost:8080`.

Terminal 2 :

```
cd /root/Cours/RansomwareRE/sample

# Préparer les fichiers de test
cd ~/Documents/VictimFiles
```

```
rm -f *.locked HOW_TO_DECRYPT.txt 2>/dev/null
for i in {1..5}; do
    echo "Important document $i" > doc_$i.txt
    echo "Photo data $i" > photo_$i.jpg
done

# Exécuter le ransomware test
cd /root/Cours/RansomwareRE/sample
rm /tmp/.crypto_* 2>/dev/null
python3 cryptolocker_test_c2.py
```

Étape 3 : Observer la Communication C2

Sortie du ransomware (Terminal 2) :

```
[*] Starting ransomware test (C2 enabled)...
[*] Victim ID: OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa
[DEBUG] Attempting to connect to C2: 127.0.0.1:8080
[DEBUG] Sending beacon: VICTIM:OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa
[DEBUG] C2 Response: OK:ENCRYPTED
[✓] C2 connection successful!
[!] Encryption complete: 10 files encrypted
[!] Victim ID: OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa
[!] See HOW_TO_DECRYPT.txt for instructions
```

Logs du serveur C2 (Terminal 1) :

```
[2024-12-11 21:46:30] [+] Connection from 127.0.0.1:35464
[2024-12-11 21:46:30] [>] Received: VICTIM:OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa
[2024-12-11 21:46:30] [!] NEW VICTIM: OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa from
127.0.0.1
[2024-12-11 21:46:30] [<] Sent: OK:ENCRYPTED
```

Étape 4 : Capture du Trafic avec tcpdump (Optionnel)

Terminal 3 :

```
sudo tcpdump -i lo -A -s 0 'port 8080' 2>&1 | tee c2_traffic.log
```

Vous verrez les paquets TCP contenant :

- Beacon : **VICTIM:OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa**
- Réponse C2 : **OK:ENCRYPTED**

Étape 5 : Arrêter le Serveur C2

Dans le Terminal 1, appuyez sur **Ctrl+C** :

```
[2024-12-11 21:47:00]
[!] Shutting down C2 server...
[2024-12-11 21:47:00] [*] Total victims: 1
[2024-12-11 21:47:00]      - OEImWHaBQ1Z0x3m89ACUZBeSYoJ4suXa: 1 beacons
from 127.0.0.1
```

Analyse de la Communication

Protocole C2 :

- 1. **Connexion TCP** : Le ransomware établit une connexion vers **127.0.0.1:8080**
- 2. **Beacon** : Envoi du message **VICTIM:<ID>\n**
- 3. **Réponse** : Le serveur répond **OK:ENCRYPTED\n**
- 4. **Fermeture** : La connexion se ferme proprement

Format du beacon :

```
VICTIM:<VICTIM_ID>
```

Format de la réponse C2 :

```
OK:ENCRYPTED
```

Différences entre les Versions

Caractéristique	cryptolocker_advanced.py	cryptolocker_test_c2.py
C2 Server	192.168.1.100:8080 (fictif)	127.0.0.1:8080 (localhost)
Connexion	Échoue silencieusement	Fonctionne avec fake_c2_server.py
Debug output	Non	Oui (messages [DEBUG])
Usage	TP principal	Tests C2 avancés

Nettoyage

```
# Arrêter le serveur C2 (Ctrl+C dans Terminal 1)

# Supprimer les fichiers de test
cd ~/Documents/VictimFiles
rm -f *.locked HOW_TO_DECRYPT.txt

# Recréer les fichiers propres
```

```
for i in {1..5}; do
    echo "Important document $i" > doc_$i.txt
    echo "Photo data $i" > photo_$i.jpg
done

# Supprimer le mutex
rm /tmp/.crypto_*
```

Ressources Complémentaires

Documentation Technique

- **Fernet Specification** : <https://github.com/fernet/spec/blob/master/Spec.md>
- **PBKDF2 (RFC 8018)** : <https://tools.ietf.org/html/rfc8018>
- **AES-CBC Mode** : <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>

Outils d'Analyse

- **IDA Pro / Ghidra** : Désassembleurs pour binaires
- **PyInstaller Extractor** : <https://github.com/extremecoders-re/pyinstxtractor>
- **Uncompyle6** : Décompilateur Python bytecode
- **Volatility** : Analyse mémoire forensique

Ransomwares Réels (à étudier, pas exécuter !)

- **WannaCry** : Exploit EternalBlue, kill switch domain
- **Locky** : Distribution via macros Office malveillantes
- **Ryuk** : Ransomware ciblé, suppression VSS
- **REvil/Sodinokibi** : Ransomware-as-a-Service (RaaS)

Base de Données de Rançongiciels

- **No More Ransom** : <https://www.nomoreransom.org/en/decryption-tools.html>
- **ID Ransomware** : <https://id-ransomware.malwarehunterteam.com/>